

Desafio Técnico

CaseCellShop

Nível Júnior | Fullstack

Empresa fictícia. Caso usado exclusivamente para fins de avaliação técnica.

CaseCellShop | Documento do Candidato

Resumo rápido

Formato: perguntas conceituais + mini-tarefa prática de checkout.

Objetivo: avaliar raciocínio técnico, clareza de solução, comunicação e uma pequena implementação, sem exigir uma solução pronta para produção.

Uso de IA: permitido e encorajado quando fizer sentido. Registre os prompts mais relevantes em um PROMPTS.md no repositório ou no final do documento de resposta.

Entrega: respostas conceituais + repositório GitHub público + PROMPTS.md, quando aplicável. Priorize clareza, simplicidade e uma entrega executável.

Stack preferencial: Node.js + TypeScript no back-end e React + TypeScript no front-end, pois essa é a stack mais próxima do nosso dia a dia.

Se preferir usar outra stack: tudo bem. Explique brevemente a escolha no README e garanta que o projeto seja simples de executar.

Serviços de apoio: use recursos locais ou simulados, como dados em memória, banco local, cache/fila local, Docker Compose, LocalStack ou mocks. Não é necessário contratar ou configurar conta real em provedor de nuvem.

Critérios de Avaliação

Importante — Método de Correção:

Serão considerados principalmente:

- clareza na análise dos problemas;
- simplicidade e coerência da solução proposta;
- tratamento de erros e estados de carregamento;
- validações mínimas no fluxo de checkout;
- comunicação de trade-offs;

- uso responsável de IA, quando aplicável.

Bem-vindo

Bem-vindo(a) ao desafio técnico da CaseCellShop! O objetivo deste teste não é apenas avaliar a sua escrita de código, mas sim a sua capacidade de entender problemas de software, pensar criticamente, propor soluções e, em uma parte menor, produzir um trecho de código que ilustre o seu raciocínio.

Contexto do case

Você foi contratado(a) como Desenvolvedor(a) na CaseCellShop, uma empresa varejista focada na venda de capinhas para celular. A empresa está passando por um período de hipercrecimento: o que antes eram milhares de acessos diários na loja virtual, hoje se transformaram em milhões.

Arquitetura atual

Componente	Descrição
ERP Central	ERP monolítico que gerencia estoque, faturamento, financeiro e contábil. É o coração da empresa.
Loja Virtual	E-commerce que consome dados (produtos, preços, estoque) diretamente do ERP via API RESTful síncrona.
Banco de Dados	ERP utiliza MySQL. Temos acesso de leitura, mas não podemos alterar rotinas, tabelas ou código interno do ERP.
Infraestrutura	Tudo hospedado em datacenter próprio (on-premise) na sede.
Monitoramento	Ferramentas que monitoram performance e enviam alertas críticos.

Os 3 problemas identificados

Com o aumento drástico de acessos, três ofensores principais foram identificados:

01 | Performance da vitrine

A vitrine demora muitos segundos para carregar produtos, frustrando clientes logo no início da jornada.

02 | Consistência de estoque

Vários clientes conseguem comprar o mesmo produto quando o estoque acaba. A empresa está vendendo itens que não possui.

03 | Resiliência do checkout

Ao finalizar a compra, a API do ERP demora para processar o pedido e gerar faturamento. A requisição sofre timeout e o cliente perde a compra.

Parte 1.A — Perguntas Conceituais

Orientação: responda em texto, tópicos ou diagramas simples. Não precisa escrever código nesta parte.

Pergunta 1 — Leitura inicial dos problemas

Para cada um dos 3 problemas identificados:

- O que você acredita estar causando o problema?
- Qual seria o impacto desse problema para o negócio ou para o cliente?
- Qual seria sua primeira hipótese de caminho para investigar ou melhorar?

Não é necessário detalhar a solução completa nesta pergunta. As próximas questões vão aprofundar infraestrutura, contrato de API, testes e uso de IA.

Pergunta 2 — Infraestrutura e serviços de apoio

De forma geral, o que você faria em relação à infraestrutura atual para suportar muitos acessos futuros sem depender diretamente do ERP em cada requisição?

- Na resposta, cite pelo menos 3 conceitos ou serviços que poderiam ajudar.
- Não precisa conhecer uma cloud específica; o importante é explicar o papel de cada item.

Pergunta 3 — SDD: Spec-Driven Development

Imagine que você precisa implementar o endpoint POST /checkout que finaliza a compra. Antes de codificar:

- Que informações esse endpoint precisa receber?
- O que ele deve devolver em caso de sucesso?
- O que ele deve devolver em caso de erro?
- Por que é importante definir esse contrato antes de escrever código?

Pergunta 4 — TDD: Test-Driven Development

Sobre testes do mesmo endpoint POST /checkout:

- Que testes você escreveria para garantir que ele funciona corretamente? Liste pelo menos 3 cenários.

- Há vantagem em escrever os testes antes de implementar a rota? Por quê?

Pergunta 5 — Uso de IA no desenvolvimento

Se você fosse implementar a solução para o Problema 2 (Furo de Estoque) usando IA:

- Que perguntas ou instruções você daria à IA?

Parte 1.B — Mini-tarefa de Código

Orientação: suba o código em um repositório público pessoal no GitHub e cole o link no final da sua resposta.

Escopo da mini-tarefa

Implemente um pequeno fluxo de checkout para compra de capinhas de celular.

O sistema deve permitir que um usuário tente comprar um produto informando uma quantidade. A aplicação deve lidar com sucesso, falhas de validação e indisponibilidade da API de forma compreensível para o usuário.

Você pode usar dados em memória. Não é necessário usar banco real, autenticação, pagamento, Docker, deploy ou layout elaborado.

O que esperamos observar na entrega

Back-end

- ☐ Existe uma API para criar uma tentativa de compra.
- ☐ A API valida entradas inválidas.
- ☐ A API diferencia cenários de sucesso e erro com respostas HTTP adequadas.
- ☐ Existe alguma representação simples de produtos e estoque.

Front-end

- ☐ Existe uma tela simples para iniciar a compra.
- ☐ A interface informa ao usuário quando a compra está em andamento.
- ☐ A interface evita ações duplicadas durante o processamento.
- ☐ A interface exibe mensagens compreensíveis em caso de sucesso ou erro.

Qualidade e entrega

- ☐ O projeto possui README explicando como rodar.
- ☐ O código está organizado de forma compreensível.
- ☐ Desejável: incluir testes automatizados.
- ☐ Bônus: registrar decisões, trade-offs ou prompts de IA utilizados.

Como entregar

- ☐ Respostas das 5 perguntas conceituais (Parte 1.A).
- ☐ Link do repositório GitHub público (Parte 1.B).
- ☐ Conteúdo do PROMPTS.md, quando aplicável.